# Summarize private documents using RAG LangChain and LLMs

July 29, 2026

## 1 Summarize Private Documents Using RAG, LangChain, and LLMs

**Estimated time needed: 45 minutes** Imagine it's your first day at an exciting new job at a fast-growing tech company, Innovatech. You're filled with a mix of anticipation and nerves, eager to make a great first impression and contribute to your team. As you find your way to your desk, decorated with a welcoming note and some company swag, you can't help but feel a surge of pride. This is the moment you've been working towards, and it's finally here.

Your manager, Alex, greets you with a warm smile. "Welcome aboard! We're thrilled to have you with us. I have sent you a folder. Inside this folder, you'll find everything you need to get up to speed on our company policies, culture, and the projects your team is working on. Please keep them private."

You thank Alex and open the folder, only to be greeted by a mountain of documents - manuals, guidelines, technical documents, project summaries, and more. It's overwhelming. You think to yourself, "How am I supposed to absorb all of this information in a short time? And they are private and I cannot just upload it to GPT to summarize them." "Why not create an agent to read and summarize them for you, and then you can just ask it?" your colleague, Jordan, suggests with an encouraging grin. You're intrigued, but uncertain; the world of large language models (LLMs) is one that you've only scratched the surface of. Sensing your hesitation, Jordan elaborates, "Imagine having a personal assistant who's not only exceptionally fast at reading but can also understand and condense the information into easy-to-digest summaries. That's what an LLM can do for you, especially when enhanced with LangChain and Retrieval-Augmented Generation (RAG) technology."

"But how do I get started? And how long will it take to set up something like that?" you ask. Jordan says, "Let's dive into a project that will not only help you tackle this immediate challenge but also equip you with a skill set that's becoming indispensable in this field."

---

So, this project steps you through the fascinating world of LLMs and RAG, starting from the basics of what these technologies are, to building a practical application that can read and summarize documents for you. By the end of this tutorial, you have a working tool capable of processing the pile of documents on your desk, allowing you to focus on making meaningful contributions to your projects sooner.

### 1.1 Table of Contents



```
</li>
<li>
    <a href="#Objectives">Objectives</a>
</li>
<li>
    <a href="#Setup">Setup</a>
    <ol>
        <li><a href="#Installing-required-libraries">Installing required libraries</a></li>
        <li><a href="#Importing-required-libraries">Importing required libraries</a></li>
    </ol>
</li>
<li>
    <a href="#Preprocessing">Preprocessing</a>
    <ol>
        <li><a href="#Load-the-document">Load the document</a></li>
        <li><a href="#Splitting-the-document-into-chunks">Splitting the document into chunks</a
        <li><a href="#Embedding-and-storing">Embedding and storing</a></li>
    </ol>
</li>
<li>
    <a href="#LLM-model-construction">LLM model construction</a>
</li>
<li>
    <a href="#Integrating-LangChain">Integrating LangChain</a>
</li>
<li>
    <a href="#Dive-deeper">Dive deeper</a>
    <ol>
        <li><a href="#Using-prompt-template">Using prompt template</a></li>
        <li><a href="#Make-the-conversation-have-memory">Make the conversation have memory</a>
        <li><a href="#Wrap-up-and-make-it-an-agent">Wrap up and make it an agent</a></li>
    </ol>
</li>
```

## 1.2 Background

### 1.2.1 What is RAG?

One of the most powerful applications enabled by LLMs is sophisticated question-answering (Q&A) chatbots. These are applications that can answer questions about specific source information. These applications use a technique known as retrieval-augmented generation (RAG). RAG is a technique for augmenting LLM knowledge with additional data, which can be your own data.

LLMs can reason about wide-ranging topics, but their knowledge is limited to public data up to the specific point in time that they were trained. If you want to build AI applications that can reason about private data or data introduced after a model's cut-off date, you must augment the knowledge of the model with the specific information that it needs. The process of bringing and inserting the appropriate information into the model prompt is known as RAG.

LangChain has several components that are designed to help build Q&A applications and RAG applications, more generally.

### 1.2.2 RAG architecture

A typical RAG application has two main components:

- **Indexing**: A pipeline for ingesting and indexing data from a source. This usually happens offline.
- **Retrieval and generation**: The actual RAG chain takes the user query at run time and retrieves the relevant data from the index, then passes that to the model.

The most common full sequence from raw data to answer looks like the following examples.

- **Indexing**

1. Load: First, you must load your data. This is done with DocumentLoaders.
2. Split: Text splitters break large `Documents` into smaller chunks. This is useful both for indexing data and for passing it into a model because large chunks are harder to search and won't fit in a model's finite context window.
3. Store: You need somewhere to store and index your splits so that they can later be searched. This is often done using a VectorStore and Embeddings model.

source

- **Retrieval and generation**

1. Retrieve: Given a user input, relevant splits are retrieved from storage using a retriever.
2. Generate: A ChatModel / LLM produces an answer using a prompt that includes the question and the retrieved data.

source

## 1.3 Objectives

After completing this lab, you will be able to:

- Master the technique of splitting and embedding documents into formats that LLMs can efficiently read and interpret.
- Know how to access and utilize different LLMs from IBM watsonx.ai, selecting the optimal model for their specific document processing needs.
- Implement different retrieval chains from LangChain, tailoring the document retrieval process to support various purposes.
- Develop an agent that uses integrated LLM, LangChain, and RAG technologies for interactive and efficient document retrieval and summarization, making conversations have memory.

---

## 1.4 Setup

For this lab, you are going to use the following libraries:

- `ibm-watsonx-ai` for using LLMs from IBM's watsonx.ai
- `LangChain` for using its different chain and prompt functions
- `Hugging Face` and `Hugging Face Hub` for their embedding methods for processing text data
- `SentenceTransformers` for transforming sentences into high-dimensional vectors
- `Chroma DB` for efficient storage and retrieval of high-dimensional text vector data
- `wget` for downloading files from remote systems

### 1.4.1 Installing required libraries

The following required libraries are **not** preinstalled in the Skills Network Labs environment. **You must run the following cell** to install them:

**Note:** The version has been pinned here to specify the version. It's recommended that you do this as well. Even though the library will be updated in the future, the library could still support this lab work.

**This might take approximately 3-5 minutes.**

As `%%capture` is used to capture the installation, you won't see the output process. But once the installation is done, you will see a number beside the cell.

[ ]:
```
%%capture
!pip install ibm-watsonx-ai==0.2.6
!pip install langchain==0.1.16
!pip install langchain-ibm==0.1.4
!pip install transformers==4.41.2
!pip install huggingface-hub==0.23.4
!pip install sentence-transformers==2.5.1
!pip install chromadb
!pip install wget==3.2
!pip install --upgrade torch --index-url https://download.pytorch.org/whl/cpu
```

After the installation of libraries is completed, restart your kernel. You can do that by clicking the **Restart the kernel** icon.

### 1.4.2 Importing required libraries

*It is recommended that you import all required libraries in one place (here):*

[ ]:
```
!pip list | grep langchain
```

[ ]:
```
# You can use this section to suppress warnings generated by your code:
def warn(*args, **kwargs):
    pass
import warnings
warnings.warn = warn
warnings.filterwarnings('ignore')

from langchain.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.vectorstores import Chroma
from langchain.embeddings import HuggingFaceEmbeddings
from langchain.chains import RetrievalQA
from langchain.prompts import PromptTemplate
from langchain.chains import ConversationalRetrievalChain
from langchain.memory import ConversationBufferMemory

from ibm_watsonx_ai.foundation_models import Model
from ibm_watsonx_ai.metanames import GenTextParamsMetaNames as GenParams
from ibm_watsonx_ai.foundation_models.utils.enums import ModelTypes,␣
 ↪DecodingMethods
from ibm_watson_machine_learning.foundation_models.extensions.langchain import␣
 ↪WatsonxLLM
import wget
```

## 1.5 Preprocessing

### 1.5.1 Load the document

The document, which is provided in a TXT format, outlines some company policies and serves as an example data set for the project.

This is the `load` step in `Indexing`.

[ ]:
```
filename = 'companyPolicies.txt'
url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/
 ↪6JDbUb_L3egv_eOkouY71A.txt'

# Use wget to download the file
wget.download(url, out=filename)
print('file downloaded')
```

After the file is downloaded and imported into this lab environment, you can use the following code to look at the document.

[ ]:
```
with open(filename, 'r') as file:
    # Read the contents of the file
    contents = file.read()
    print(contents)
```

From the content, you see that the document discusses nine fundamental policies within a company.

### 1.5.2 Splitting the document into chunks

In this step, you are splitting the document into chunks, which is basically the `split` process in `Indexing`.

`LangChain` is used to split the document and create chunks. It helps you divide a long story (document) into smaller parts, which are called `chunks`, so that it's easier to handle.

For the splitting process, the goal is to ensure that each segment is as extensive as if you were to count to a certain number of characters and meet the split separator. This certain number is called `chunk size`. Let's set 1000 as the chunk size in this project. Though the chunk size is 1000, the splitting is happening randomly. This is an issue with LangChain. `CharacterTextSplitter` uses `\n\n` as the default split separator. You can change it by adding the `separator` parameter in the `CharacterTextSplitter` function; for example, `separator="\n"`.

[ ]:
```
loader = TextLoader(filename)
documents = loader.load()
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
texts = text_splitter.split_documents(documents)
print(len(texts))
```

From the ouput of print, you see that the document has been split into 16 chunks

### 1.5.3 Embedding and storing

This step is the `embed` and `store` processes in `Indexing`.

In this step, you're taking the pieces of the story, your "chunks," converting the text into numbers, and making them easier for your computer to understand and remember by using a process called "embedding." Think of embedding like giving each chunk its own special code. This code helps the computer quickly find and recognize each chunk later on.

You do this embedding process during a phase called "Indexing." The reason why is to make sure that when you need to find specific information or details within your larger document, the computer can do so swiftly and accurately.

The following code creates a default embedding model from Hugging Face and ingests them to Chromadb.

When it's completed, print "document ingested".

[ ]:
```
embeddings = HuggingFaceEmbeddings()
docsearch = Chroma.from_documents(texts, embeddings)  # store the embedding in␣
 ↪docsearch using Chromadb
print('document ingested')
```

Up to this point, you've been performing the `Indexing` task. The next step is the `Retrieval` task.

### 1.6 LLM model construction

In this section, you'll build an LLM model from IBM watsonx.ai.

First, define a model ID and choose which model you want to use. There are many other model options. Refer to Foundation Models for other model options. This tutorial uses the `granite` model as an example.

[ ]:
```
model_id = 'ibm/granite-3-8b-instruct'
```

Define parameters for the model.

The decoding method is set to `greedy` to get a deterministic output.

For other commonly used parameters, you can refer to Foundation model parameters: decoding and stopping criteria.

[ ]:
```
parameters = {
    GenParams.DECODING_METHOD: DecodingMethods.GREEDY,
    GenParams.MIN_NEW_TOKENS: 130, # this controls the minimum number of tokens␣
 ↪in the generated output
    GenParams.MAX_NEW_TOKENS: 256,  # this controls the maximum number of␣
 ↪tokens in the generated output
    GenParams.TEMPERATURE: 0.5 # this randomness or creativity of the model's␣
 ↪responses
}
```

Define `credentials` and `project_id`, which are necessary parameters to successfully run LLMs from watsonx.ai.

(Keep `credentials` and `project_id` as they are now so that you do not need to create your own keys to run models. This supports you in running the model inside this lab environment. However, if you want to run the model locally, refer to this tutorial for creating your own keys.

### 1.7 API Disclaimer

This lab uses LLMs provided by **Watsonx.ai**. This environment has been configured to allow LLM use without API keys so you can prompt them for **free (with limitations)**. With that in mind, if you wish to run this notebook **locally outside** of Skills Network's JupyterLab environment, you will have to **configure your own API keys**. Please note that using your own API keys means that you will incur personal charges.

#### 1.7.1 Running Locally

If you are running this lab locally, you will need to configure your own API keys. This lab uses the `WatsonxLLM` module from `IBM`. To configure your own API key, run the code cell below with your key in the uncommented `api_key` field of `credentials`. **DO NOT** uncomment the `api_key` field if you aren't running locally, it will causes errors.

[ ]:
```
credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
    # "api_key": "your api key here"
    # uncomment above when running locally
}

project_id = "skills-network"
```

Wrap the parameters to the model.

[ ]:
```
model = Model(
    model_id=model_id,
    params=parameters,
    credentials=credentials,
    project_id=project_id
)
```

Build a model called `flan_ul2_llm` from watsonx.ai.

[ ]:
```
flan_ul2_llm = WatsonxLLM(model=model)
```

This completes the `LLM` part of the `Retrieval` task.

### 1.8 Integrating LangChain

LangChain has a number of components that are designed to help retrieve information from the document and build question-answering applications, which helps you complete the `retrieve` part of the `Retrieval` task.

In the following steps, you create a simple Q&A application over the document source using LangChain's `RetrievalQA`.

Then, you ask the query "what is mobile policy?"

[ ]:
```
qa = RetrievalQA.from_chain_type(llm=flan_ul2_llm,
                                 chain_type="stuff",
                                 retriever=docsearch.as_retriever(),
                                 return_source_documents=False)
query = "what is mobile policy?"
qa.invoke(query)
```

From the response, it seems fine. The model's response is the relevant information about the mobile policy from the document.

Now, try to ask a more high-level question.

[ ]:
```
qa = RetrievalQA.from_chain_type(llm=flan_ul2_llm,
                                 chain_type="stuff",
                                 retriever=docsearch.as_retriever(),
                                 return_source_documents=False)
query = "Can you summarize the document for me?"
```

```
qa.invoke(query)
```

So, you can try with any other model. If so then, You should do the model construction again.

[ ]:
```
model_id = 'ibm/granite-3-8b-instruct'

parameters = {
    GenParams.DECODING_METHOD: DecodingMethods.GREEDY,
    GenParams.MAX_NEW_TOKENS: 256,  # this controls the maximum number of␣
 ↪tokens in the generated output
    GenParams.TEMPERATURE: 0.5 # this randomness or creativity of the model's␣
 ↪responses
}

credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
}

project_id = "skills-network"

model = Model(
    model_id=model_id,
    params=parameters,
    credentials=credentials,
    project_id=project_id
)

llama_3_llm = WatsonxLLM(model=model)
```

Try the same query again on this model.

[ ]:
```
qa = RetrievalQA.from_chain_type(llm=llama_3_llm,
                                 chain_type="stuff",
                                 retriever=docsearch.as_retriever(),
                                 return_source_documents=False)
query = "Can you summarize the document for me?"
qa.invoke(query)
```

Now, you've created a simple Q&A application for your own document. Congratulations!

### 1.9 Dive deeper

This section dives deeper into how you can improve this application. You might want to ask "How to add the prompt in retrieval using LangChain?"

You use prompts to guide the responses from an LLM the way you want. For instance, if the LLM is uncertain about an answer, you instruct it to simply state, "I do not know," instead of attempting to generate a speculative response.

Let's see an example.

```
[ ]: qa = RetrievalQA.from_chain_type(llm=flan_ul2_llm,
                                      chain_type="stuff",
                                      retriever=docsearch.as_retriever(),
                                      return_source_documents=False)
     query = "Can I eat in company vehicles?"
     qa.invoke(query)
```

As you can see, the query is asking something that does not exist in the document. The LLM responds with information that actually is not true. You don't want this to happen, so you must add a prompt to the LLM.

### 1.9.1 Using prompt template

In the following code, you create a prompt template using `PromptTemplate`.

`context` and `question` are keywords in the RetrievalQA, so LangChain can automatically recognize them as document content and query.

```
[ ]: prompt_template = """Use the information from the document to answer the␣
      ↪question at the end. If you don't know the answer, just say that you don't␣
      ↪know, definately do not try to make up an answer.

     {context}

     Question: {question}
     """

     PROMPT = PromptTemplate(
         template=prompt_template, input_variables=["context", "question"]
     )

     chain_type_kwargs = {"prompt": PROMPT}
```

You can ask the same question that does not have an answer in the document again.

```
[ ]: qa = RetrievalQA.from_chain_type(llm=llama_3_llm,
                                      chain_type="stuff",
                                      retriever=docsearch.as_retriever(),
                                      chain_type_kwargs=chain_type_kwargs,
                                      return_source_documents=False)

     query = "Can I eat in company vehicles?"
     qa.invoke(query)
```

From the answer, you can see that the model responds with "don't know".

### 1.9.2 Make the conversation have memory

Do you want your conversations with an LLM to be more like a dialogue with a friend who remembers what you talked about last time? An LLM that retains the memory of your previous

exchanges builds a more coherent and contextually rich conversation.

Take a look at a situation in which an LLM does not have memory.

You start a new query, "What I cannot do in it?". You do not specify what "it" is. In this case, "it" means "company vehicles" if you refer to the last query.

[ ]:
```
query = "What I cannot do in it?"
qa.invoke(query)
```

From the response, you see that the model does not have the memory because it does not provide the correct answer, which is something related to "smoking is not permitted in company vehicles."

To make the LLM have memory, you introduce the `ConversationBufferMemory` function from LangChain.

[ ]:
```
memory = ConversationBufferMemory(memory_key = "chat_history", return_message =␣
↪True)
```

Create a `ConversationalRetrievalChain` to retrieve information and talk with the LLM.

[ ]:
```
qa = ConversationalRetrievalChain.from_llm(llm=llama_3_llm,
                                           chain_type="stuff",
                                           retriever=docsearch.as_retriever(),
                                           memory = memory,
                                           get_chat_history=lambda h : h,
                                           return_source_documents=False)
```

Create a `history` list to store the chat history.

[ ]:
```
history = []
```

[ ]:
```
query = "What is mobile policy?"
result = qa.invoke({"question":query}, {"chat_history": history})
print(result["answer"])
```

Append the previous query and answer to the history.

[ ]:
```
history.append((query, result["answer"]))
```

[ ]:
```
query = "List points in it?"
result = qa({"question": query}, {"chat_history": history})
print(result["answer"])
```

Append the previous query and answer to the chat history again.

[ ]:
```
history.append((query, result["answer"]))
```

[ ]:
```
query = "What is the aim of it?"
result = qa({"question": query}, {"chat_history": history})
print(result["answer"])
```

### 1.9.3 Wrap up and make it an agent

The following code defines a function to make an agent, which can retrieve information from the document and has the conversation memory.

[ ]:
```
def qa():
    memory = ConversationBufferMemory(memory_key = "chat_history",␣
 ↪return_message = True)
    qa = ConversationalRetrievalChain.from_llm(llm=llama_3_llm,
                                               chain_type="stuff",
                                               retriever=docsearch.
 ↪as_retriever(),
                                               memory = memory,
                                               get_chat_history=lambda h : h,
                                               return_source_documents=False)
    history = []
    while True:
        query = input("Question: ")

        if query.lower() in ["quit","exit","bye"]:
            print("Answer: Goodbye!")
            break

        result = qa({"question": query}, {"chat_history": history})

        history.append((query, result["answer"]))

        print("Answer: ", result["answer"])
```

Run the function.

Feel free to answer questions for your chatbot. For example:

*What is the smoking policy? Can you list all points of it? Can you summarize it?*

To **stop** the agent, you can type in 'quit', 'exit', 'bye'. Otherwise you cannot run other cells.

[ ]:
```
qa()
```

Congratulations! You have finished the project. Following are three exercises to help you to extend your knowledge.

## 2 Exercises

### 2.0.1 Exercise 1: Work on your own document

You are welcome to use your own document to practice. Another document has also been prepared that you can use for practice. Can you load this document and make the LLM read it for you? Here is the URL to the document: https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/XVnuuEg94sAE4S_xAsGxBA.txt

[ ]:
```
# Add your code here
```

Click here for solution

```
filename = 'stateOfUnion.txt'
url = 'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/XVnuuEg94sAE4S_xAsGx

wget.download(url, out=filename)
print('file downloaded')
```

### 2.0.2 Exercise 2: Return the source from the document

Sometimes, you not only want the LLM to summarize for you, but you also want the model to return the exact content source from the document to you for reference. Can you adjust the code to make it happen?

[ ]:
```
# Add your code
```

Click here for a hint

All you must do is change the return_source_documents to True when you create the chain. And when you print, print the ['source_documents'][0]

```
qa = RetrievalQA.from_chain_type(llm=llama_3_llm, chain_type="stuff", retriever=docsearch.as_r
query = "Can I smoke in company vehicles?"
results = qa.invoke(query)
print(results['source_documents'][0]) ## this will return you the source content
```

### 2.0.3 Exercise 3: Use another LLM model

IBM watsonx.ai also has many other LLM models that you can use; for example, `mistralai/mistral-small-3-1-24b-instruct-2503`, an open-source model from Mistral AI. Can you change the model to see the difference of the response?

[ ]:
```
# Add your code here
```

Click here for a hint

To use a different LLM, go to the cell where the `model_id` is specified and replace the current `model_id` with the following code. Expect different results and performance when using different LLMs:

```
model_id = 'mistralai/mistral-small-3-1-24b-instruct-2503'
```

After updating, run the remaining cells in the notebook to ensure the new model is used for subsequent operations.

## 2.1 Authors

Kang Wang Kang Wang is a Data Scientist Intern in IBM. He is also a PhD Candidate in the University of Waterloo.

Faranak Heidari Faranak Heidari is a Data Scientist Intern in IBM with a strong background in applied machine learning. Experienced in managing complex data to establish business insights and foster data-driven decision-making in complex settings such as healthcare. She is also a PhD candidate at the University of Toronto.

### 2.1.1 Other Contributors

Sina Nazeri I am grateful to have had the opportunity to work as a Research Associate, Ph.D., and IBM Data Scientist. Through my work, I have gained experience in unraveling complex data structures to extract insights and provide valuable guidance.

Wojciech "Victor" Fulmyk Wojciech "Victor" Fulmyk is a Data Scientist at IBM and a Ph.D. candidate in Economics at the University of Calgary.

```
{## Change Log}

{|Date (YYYY-MM-DD)|Version|Changed By|Change Description||-|-|-|-||2024-03-22|0.1|Kang Wang|Create the Project|}
```